

Departamento Electrónica — Técnicas Digitales II

Actividad de Formación Práctica N°5 — LAB

Tema: Antirrebote por máquina de estados.

Profesor: Ing. Rubén Darío Mansilla **ATTP:** Ing. Lucas Abdala

AFP 5 — Grupo 3 — TDII 2026

Integrantes:

- Plaate Iván
- *(completar compañeros)*
- *(completar compañeros)*

1. Introducción

El presente informe documenta el desarrollo de la AFP N°5 de laboratorio, cuyo objetivo fue implementar un **driver de antirrebote para pulsadores** (`API_debounce`) mediante una **máquina de estados finitos (MEF)**, e integrarlo en las aplicaciones de las prácticas anteriores sobre la placa **STM32 NUCLEO-F429ZI**.

Los pulsadores mecánicos producen un tren de pulsos espurios ("rebote") al ser accionados, de duración típica de hasta unas decenas de milisegundos. Sin tratamiento, cada pulsación puede interpretarse como varios eventos falsos. El driver implementado filtra el rebote por software exigiendo que cada flanco permanezca estable durante **40 ms** antes de validarlo.

La práctica completa el esquema acumulativo de la materia: cada proyecto carga los tres drivers — `API_GPIO` (AFP3), `API_delay` (AFP4) y `API_debounce` (AFP5) — sin modificarlos entre prácticas.

2. Desarrollo del driver `API_debounce`

2.1. Estructura y dependencias

```
Drivers/  
└─ API/  
    ├── Inc/  
    │   ├── API_GPIO.h  
    │   ├── API_delay.h  
    │   └─ API_debounce.h  
    └─ Src/  
        └─ API_GPIO.c
```

```
└─ API_delay.c
└─ API_debounce.c
```

El driver reutiliza `API_delay` internamente: la validación temporal de cada flanco se realiza con una instancia `delay_t` de 40 ms, sin usar ningún retardo bloqueante.

2.2. La máquina de estados

```
typedef enum
{
    BUTTON_UP,                /* Pulsador liberado (estado inicial) */
    BUTTON_FALLING,           /* Posible pulsacion: esperando validar 40 ms */
    BUTTON_DOWN,              /* Pulsador presionado (validado) */
    BUTTON_RISING,            /* Posible liberacion: esperando validar 40 ms */
} debounceState_t;
```

El funcionamiento de la MEF:

Estado actual	Condición	Próximo estado / acción
BUTTON_UP	Lectura en alto (presionado)	BUTTON_FALLING — arma el retardo de 40 ms
BUTTON_FALLING	Cumplidos 40 ms y sigue presionado	BUTTON_DOWN — pulsación validada , dispara <code>buttonPressed()</code>
BUTTON_FALLING	Cumplidos 40 ms y ya liberado	BUTTON_UP — era rebote, se descarta
BUTTON_DOWN	Lectura en bajo (liberado)	BUTTON_RISING — arma el retardo de 40 ms
BUTTON_RISING	Cumplidos 40 ms y sigue liberado	BUTTON_UP — liberación validada , dispara <code>buttonReleased()</code>
BUTTON_RISING	Cumplidos 40 ms y aún presionado	BUTTON_DOWN — era rebote, sigue presionado

La validación exige que la lectura se mantenga **estable durante 40 ms completos** (`DEBOUNCE_DELAY`): cualquier pulso más corto que ese tiempo se descarta automáticamente.

2.3. Funciones exportadas

Función	Descripción
<code>debounceFSM_init()</code>	Inicializa la MEF en <code>BUTTON_UP</code> y arma el retardo de validación
<code>debounceFSM_update(bool_t buttonRead)</code>	Actualiza la MEF; debe llamarse en cada iteración del lazo principal con la lectura lógica actual del pulsador
<code>readKey()</code>	Devuelve <code>true</code> una sola vez por cada pulsación validada (evento tipo <i>one-shot</i>); la marca interna se reinicia al leerla

Función	Descripción
<code>buttonPressed()</code>	Acción ejecutada al validarse una pulsación — la implementa cada aplicación
<code>buttonReleased()</code>	Acción ejecutada al validarse una liberación — la implementa cada aplicación

2.4. Portabilidad del driver

El driver no lee ningún pin ni acciona ningún LED directamente. La aplicación le **inyecta** la lectura lógica del pulsador en cada iteración:

```
debounceFSM_update(readButton_GPIO(userButton));
```

y define sus propias acciones de flanco implementando `buttonPressed()` y `buttonReleased()` en el `main.c`, típicamente con funciones de `API_GPIO`. De esta forma el driver sirve para **cualquier pulsador en cualquier puerto y cualquier placa**, y una misma instancia del driver puede reutilizarse en aplicaciones con acciones completamente distintas. Esta fue la corrección de arquitectura aplicada respecto de implementaciones de referencia, que suelen incluir el puerto `GPIOB` y las llamadas `HAL_GPIO_*` dentro de la propia MEF.

3. Aplicaciones desarrolladas

Se crearon cuatro aplicaciones con el formato de nombre `App_5_Y_Grupo_3_2026`. Todas cargan los tres drivers acumulados (`API_GPIO` + `API_delay` + `API_debounce`) y ninguna lee el pulsador por fuera de la MEF.

3.1. App_5_1 — Validación del antirrebote

Aplicación de demostración de la MEF: en cada iteración del lazo se actualiza la máquina con la lectura del pulsador, y las acciones de flanco implementadas en la aplicación son:

- `buttonPressed()` → invierte el estado de **LD1** (verde).
- `buttonReleased()` → invierte el estado de **LD3** (rojo).

Permite observar directamente la ventana de validación de 40 ms: mantener el pulsador presionado no genera eventos repetidos, y los pulsos espurios cortos (por ejemplo, al rozar el botón) son filtrados.

3.2. App_5_2 — Inversión de sentido con pulsación validada

Refactor de `App_4_2`: secuencia circular `LD1 → LD2 → LD3` (200 ms encendido / 200 ms apagado) con retardos no bloqueantes. `buttonPressed()` invierte el sentido de la secuencia; ahora la inversión solo ocurre una vez por pulsación **validada**, sin posibilidad de dobles cambios por rebote.

3.3. App_5_3 — Cuatro secuencias con cambio validado

Refactor de App_4_3: el pulsador alterna las cuatro secuencias (secuencial 150 ms / simultáneo 300 ms / periodos independientes 100-300-600 ms / LD1+LD3 con LD2 inverso 150 ms). `buttonPressed()` avanza a la siguiente secuencia y reinicia el estado de la aplicación. La secuencia 3 mantiene sus tres instancias independientes de `delay_t`.

3.4. App_5_4 — Frecuencia variable con cambio validado

Refactor de App_4_4: los tres LEDs parpadean juntos y `buttonPressed()` cicla el tiempo de alternancia (100 → 250 → 500 → 1000 ms) aplicándolo con `delayWrite()` sobre el retardo en curso.

4. Observaciones

- El driver `API_debounce` es **totalmente independiente del hardware**: recibe la lectura por parámetro y las acciones las implementa cada aplicación. Esto respeta la regla de la materia de que los drivers portables no contienen pines ni llamadas HAL concretas.
- `readKey()` funciona como evento *one-shot*: aunque el programa lo consulte continuamente, devuelve `true` una sola vez por pulsación validada.
- Las funciones `buttonPressed()` / `buttonReleased()` se declaran en el driver pero su cuerpo vive en cada `main.c`. En las aplicaciones donde solo interesa el flanco de pulsación (App_5_2 a App_5_4), `buttonReleased()` queda implementada sin acción asociada.
- El antirrebote es por software y no bloqueante: la MEF se actualiza en cada iteración del lazo y la validación usa `API_delay`, por lo que el programa nunca se detiene esperando los 40 ms.
- En el hardware se verificó que los "golpes" rápidos al botón (duración menor a 40 ms) no generan eventos, y que presionar y soltar produce exactamente una pulsación y una liberación.

5. Repositorio

Repositorio grupal en GitHub:

https://github.com/ivanplaate-oss/Grupo_3_TDII_2026

Las aplicaciones se encuentran en la carpeta `AFP_5_TDII_2026/`:

- `AFP_5_TDII_2026/App_5_1_Grupo_3_2026` — validación del antirrebote (LD1/LD3)
- `AFP_5_TDII_2026/App_5_2_Grupo_3_2026` — inversión de sentido
- `AFP_5_TDII_2026/App_5_3_Grupo_3_2026` — cuatro secuencias alternadas
- `AFP_5_TDII_2026/App_5_4_Grupo_3_2026` — frecuencia de parpadeo variable

6. Conclusiones

Se implementó un driver de antirrebote por máquina de estados finitos, portable y no bloqueante, y se lo integró en las cuatro aplicaciones junto con los drivers `API_GPIO` y `API_delay` desarrollados en las prácticas anteriores, verificando el funcionamiento correcto en la placa NUCLEO-F429ZI.

El diseño por inyección de la lectura y de las acciones de flanco permite reutilizar el mismo driver con cualquier pulsador y cualquier acción, completando el esquema de drivers acumulativos de la materia:

tres capas de servicio (GPIO → retardos → antirrebote) que la aplicación combina sin acceder directamente a la HAL.